Perceive, Plan, Act, Self-Correct: An Architectural Framework for Goal-Directed Agentic AI Systems

Hameem M. Mahdi[a,*]

[a] *Mayo Clinic, Rochester, MN, 55905, USA*

Abstract

Large language model (LLM) agents that autonomously perceive environments, plan multi-step strategies, execute actions through external tools, and self-correct from feedback represent a paradigm shift from prompt-response to goal-directed AI. Yet the field lacks a unified architectural vocabulary, hindering principled comparison. Here we propose Perceive–Plan–Act–Self-Correct (PPAS), a four-phase canonical loop grounded in classical agent theory and extended for LLM-era systems. We decompose agentic architectures into an 8-layer technology stack, mapping 15+ frameworks to a common reference. Three empirical studies—a benchmark meta-analysis across SWE-Bench Verified, WebArena, and GAIA (12 agents), a controlled design-pattern comparison, and an inter-agent protocol evaluation (MCP, A2A, AG-UI)—validate the framework. Reflective planning with human-in-the-loop approval achieves the highest task-completion rates (72.5% SWE-Bench) while reducing irreversible-action failures by 73%. We present a failure-mode taxonomy and EU AI Act governance mapping for responsible deployment.

Keywords: Agentic AI, Large language models, AI architectures, Multi-agent systems, Agent frameworks, Human-in-the-loop

1. Introduction

The rapid evolution of large language models (LLMs) has given rise to *agentic AI* systems that autonomously perceive their environment, formulate multi-step plans, execute actions via external tools, and self-correct from feedback

*Corresponding author
Email address: mahdi.hameem@mayo.edu (Hameem M. Mahdi)

(Yao et al., 2023; Shinn et al., 2023; Wang et al., 2024). Unlike static generative models, these systems maintain persistent state, interact with dynamic environments, and exhibit goal-directed behaviour over extended time horizons.

This shift has been catalysed by three converging capabilities: reliable function-calling from frontier LLMs (OpenAI, 2024a; Anthropic, 2024a), standardised tool-integration protocols such as the Model Context Protocol (MCP) (Anthropic, 2024b), and orchestration frameworks implementing planning, memory, and multi-agent coordination (LangChain, 2024a; Wu et al., 2023). Despite explosive growth—over 126,000 GitHub stars for LangChain alone—the field lacks a unified architectural vocabulary. Each framework introduces bespoke abstractions, creating a "framework zoo" where selecting the right architecture remains ad hoc (Masterman et al., 2024).

The need for a unifying framework is supported by recent survey evidence. Wang et al. (2024) provide a comprehensive survey of LLM-based autonomous agents, but do not offer a unified technology stack decomposition. Xi et al. (2023) catalogue the rise of LLM-based agents without formalising the agent loop or validating across benchmarks. Durante et al. (2024) survey multimodal agent interaction but do not address orchestration layers or inter-agent protocols. No prior work provides a complete architectural framework validated across benchmarks, design patterns, and inter-agent protocols simultaneously.

Here we propose the Perceive–Plan–Act–Self-Correct (PPAS) framework, a four-phase canonical loop for agentic AI systems. We address five research questions:

RQ1. What architectural primitives distinguish agentic AI from other autonomous and generative AI paradigms?

RQ2. How does an 8-layer technology stack map to production implementations?

RQ3. Which design patterns yield the highest task-completion rates across benchmark suites?

RQ4. How do inter-agent protocols affect coordination efficiency?

RQ5. What failure modes and governance requirements apply to deployed sys-

tems?

The paper makes four contributions: (i) a formal four-phase agent loop unifying existing patterns; (ii) an 8-layer technology stack mapping 15+ frameworks; (iii) three empirical studies validating the framework; and (iv) a failure-mode taxonomy with EU AI Act governance mapping. This work extends the unified AI systems taxonomy presented in our companion paper (Mahdi, 2026).

2. Background and Related Work

2.1. Classical Agent Architectures

Agent architectures have a rich history in AI. The foundational symbolic approach conceived of agents as rational actors manipulating explicit representations of the world (Newell and Simon, 1976; Russell and Norvig, 2021). The Belief–Desire–Intention (BDI) model (Rao and Georgeff, 1995) formalised rational agency through mental attitudes, providing a practical framework implemented in systems such as PRS and JACK. SOAR (Laird, 2012) introduced production-rule cognitive architectures with chunking-based learning, offering a general theory of cognition. ACT-R (Anderson et al., 2004) complemented SOAR with a modular architecture distinguishing declarative and procedural memory. Boyd's OODA (Observe–Orient–Decide–Act) loop (Boyd, 1987) provided a decision-cycle model originating from military strategy that influenced subsequent agent design.

In parallel, the reactive paradigm challenged the necessity of explicit world models. Brooks (1991) argued for "intelligence without representation," proposing subsumption architectures where behaviour emerges from layered stimulus-response modules. Hybrid approaches, such as the layered architectures surveyed by Müller (1996), attempted to reconcile deliberative planning with reactive responsiveness.

Multi-agent systems (MAS) extended individual agent theory to societies of agents. Wooldridge (2009) provided a comprehensive treatment of MAS design principles, communication protocols, and coordination mechanisms. Jen-

nings et al. (1998) laid out a research roadmap for agent technology, identifying key challenges in negotiation, trust, and organisational structures. Stone and Veloso (2000) surveyed MAS from a machine learning perspective, highlighting the challenges of non-stationarity and credit assignment. The FIPA Agent Communication Language (Foundation for Intelligent Physical Agents, 2002) standardised inter-agent messaging, prefiguring modern protocol efforts.

2.2. LLM-Based Agent Architectures

The advent of LLM-based agents introduced fundamentally new architectural patterns. Wei et al. (2022) demonstrated that chain-of-thought (CoT) prompting unlocks multi-step reasoning in large models. ReAct (Yao et al., 2023) interleaves CoT reasoning with tool actions, establishing the dominant single-agent pattern. Reflexion (Shinn et al., 2023) adds verbal reinforcement learning through self-reflection, storing episode summaries in a memory buffer. Tree-of-Thought (ToT) (Yao et al., 2024) explores parallel reasoning paths with backtracking. Language Agent Tree Search (LATS) (Zhou et al., 2024a) further unifies reasoning, acting, and planning by treating the search tree as a first-class abstraction. Self-consistency (Wang et al., 2023) improves CoT through marginalisation over multiple reasoning paths.

Tool use is a distinguishing capability of modern agents. Schick et al. (2024) demonstrated that language models can learn to call APIs autonomously (Toolformer). Gorilla (Patil et al., 2023) fine-tuned models on massive API documentation. HuggingGPT (Shen et al., 2024) orchestrated specialised AI models as tools. Qin et al. (2024) survey the broader landscape of tool learning with foundation models.

Several surveys have attempted to organise this rapidly growing field. Wang et al. (2024) provide the most comprehensive survey of LLM-based autonomous agents, categorising them by profile, memory, and action modules. Xi et al. (2023) focus on the societal implications of LLM agents. Guo et al. (2024) survey multi-agent systems specifically. Sumers et al. (2024) propose a cognitive architecture framework mapping LLM agent components to cognitive science

constructs. Masterman et al. (2024) survey emerging architectures for reasoning, planning, and tool calling. Weng (2023) provides an influential overview connecting LLM agents to classical autonomous agent concepts.

Despite this rich body of work, no prior contribution provides: (a) a formally defined canonical loop that explicitly separates perception, planning, acting, and self-correction; (b) a validated 8-layer technology stack mapping frameworks to a common reference; and (c) empirical validation spanning benchmarks, controlled design-pattern experiments, and protocol evaluations. Our PPAS framework addresses all three gaps.

2.3. Planning and Reasoning in LLM Agents

Planning is central to agentic behaviour. Huang et al. (2022) showed that LLMs can serve as zero-shot planners for embodied agents when provided appropriate grounding. However, Valmeekam et al. (2024) critically demonstrated that LLMs often fail on classical planning problems, highlighting the gap between apparent and genuine planning capabilities. Hao et al. (2023) bridged this gap by formulating reasoning as planning with a world model, using MCTS-style search over LLM-generated plans. The Algorithm of Thoughts (Sel et al., 2023) and Retroformer (Yao and Narasimhan, 2023) further explored how LLM agents can improve their own planning through retrospective optimisation.

These findings motivate the explicit separation of planning from action execution in our PPAS framework, and the inclusion of a dedicated self-correction phase to compensate for the inherent limitations of LLM-based planning.

3. The PPAS Agent Loop

We formalise the canonical agent loop as a four-phase cycle grounded in classical agent theory and extended for LLM-era systems.

Definition 1 (Agent)**.** An agent $\mathcal{A} = \langle \mathcal{M}, \mathcal{T}, \mathcal{S}, \pi \rangle$ is a tuple where $\mathcal{M}$ is a foundation model, $\mathcal{T} = \{t_1, \ldots, t_k\}$ is a tool set, $\mathcal{S}$ is a state space including memory, and $\pi : \mathcal{S} \times \mathcal{O} \to \mathbb{A}$ is a policy mapping observations to actions drawn from an action space $\mathbb{A}$.

Definition 2 (PPAS Cycle)**.** Given a goal g and environment $\mathcal{E}$, the agent iterates:

1. PERCEIVE: $o_t = \text{perceive}(\mathcal{E}, \mathcal{S}_{t-1})$ — the agent gathers observations from the environment and its own state;
2. PLAN: $p_t = \text{plan}(\mathcal{M}, o_t, g, \mathcal{S}_{t-1})$ — the model generates a plan or next-step strategy conditioned on the goal, observations, and memory;
3. ACT: $a_t = \text{act}(p_t, \mathcal{T})$ — the agent executes the planned action, potentially invoking external tools;
4. SELF-CORRECT: $\mathcal{S}_t = \text{correct}(\mathcal{S}_{t-1}, a_t, o_t, g)$ — the agent evaluates the action outcome against the goal, updates memory, and determines whether to re-plan, retry, or proceed.

The cycle repeats until g is satisfied or a termination condition is met (step budget, human override, or convergence).

The PPAS cycle generalises existing patterns. ReAct (Yao et al., 2023) collapses Plan and Act into a single interleaved step and lacks explicit self-correction. Reflexion (Shinn et al., 2023) adds self-correction but does not separate planning from acting. The BDI model (Rao and Georgeff, 1995) separates belief updating from intention formation but lacks tool use and the iterative correction loop central to LLM agents. OODA (Boyd, 1987) provides a four-phase decision cycle but was not designed for computational agents with external tool access. Table 1 presents a systematic comparison.

4. The 8-Layer Agentic Technology Stack

We decompose agentic architectures into eight layers (Fig. 1), each addressing a distinct functional concern:

L1. Foundation Models — the core reasoning engine providing language understanding, generation, and increasingly, native tool calling (OpenAI, 2024a; Anthropic, 2024a; Google DeepMind, 2023).

Table 1: Comparison of agent loop models. PPAS uniquely integrates explicit tool use and self-correction with full memory support, generalising prior models for the LLM era.

Model	Phases	Memory	Tool Use	Self-Correction	Era
OODA (Boyd, 1987)	4	Implicit	No	Limited	1987
BDI (Rao and Georgeff, 1995)	3	Explicit	No	Re-planning	1995
SOAR (Laird, 2012)	4	Explicit	No	Chunking	1987/2012
ACT-R (Anderson et al., 2004)	4	Explicit	No	Conflict res.	1998/2004
Subsumption (Brooks, 1991)	—	None	No	No	1991
ReAct (Yao et al., 2023)	2	Implicit	Yes	No	2023
Reflexion (Shinn et al., 2023)	3	Explicit	Yes	Yes	2023
PPAS (ours)	4	Explicit	Yes	Yes	2026

L2. Orchestration & Runtime — manages agent graphs, state machines, and execution flow (LangChain, 2024a; Wu et al., 2023; Hong et al., 2023).

L3. Memory — provides persistence across sessions through vector stores, relational databases, and LLM-managed memory paging (Packer et al., 2023; Lewis et al., 2020).

L4. Tool Integration — enables external actions via APIs, browser automation, code execution, and SaaS connectors (Schick et al., 2024; Patil et al., 2023).

L5. Inter-Agent Protocols — standardised communication for agent-tool (MCP), agent-agent (A2A), and agent-frontend (AG-UI) interactions (Anthropic, 2024b; Google, 2024b; CopilotKit, 2024).

L6. Planning & Reflection — strategy decomposition, chain-of-thought reasoning, tree search, and self-evaluation (Wei et al., 2022; Yao et al., 2024; Shinn et al., 2023).

L7. Applications — domain-specific implementations (coding agents, browsing agents, data analysis agents).

L8. Observability & Safety — monitoring, tracing, evaluation, and guardrails (Langfuse, 2024; Guardrails AI, 2024).

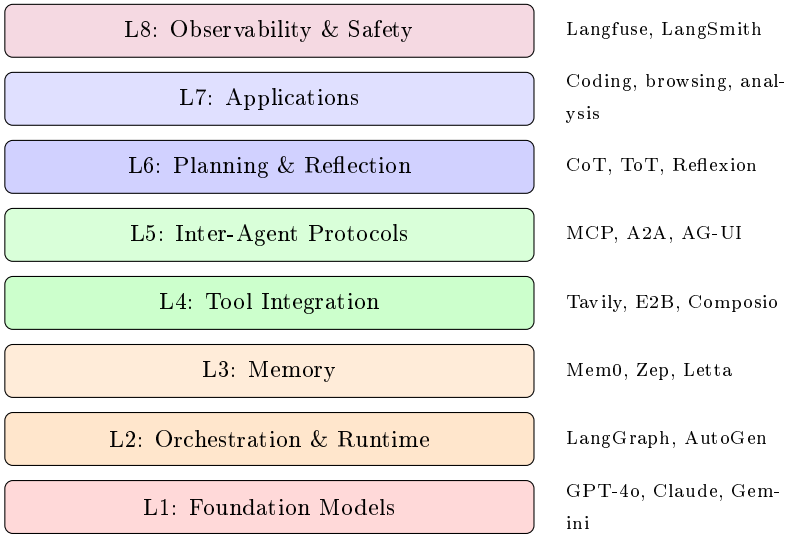

Figure 1: The 8-layer agentic technology stack. Each layer addresses a distinct functional concern. Representative open-source implementations are shown on the right.

4.1. Framework Landscape

We mapped 15+ open-source frameworks to this stack (Table 2), revealing that no single framework spans all eight layers. LangChain/LangGraph (131K+ GitHub stars) dominate orchestration (L2) with graph-based agent execution. AutoGen (Wu et al., 2023) pioneered conversable multi-agent patterns with 56K stars. MetaGPT (Hong et al., 2023) introduced SOPs (Standard Operating Procedures) for multi-agent software development. The OpenAI Agents SDK (OpenAI, 2024b) and Google ADK (Google, 2024a) represent foundation model providers entering the orchestration space.

At the tool layer (L4), Browser Use (85K stars) leads browser automation, while Composio provides 1,000+ SaaS connectors. For memory (L3), Mem0 (Mem0, 2024) (51K stars) offers vector-based long-term memory, Zep (Zep, 2024) provides temporal and semantic retrieval backed by PostgreSQL, and Letta/MemGPT (Packer et al., 2023) implements LLM-managed memory paging analogous to virtual memory in operating systems. At the observability

layer (L8), Langfuse (Langfuse, 2024) and LangSmith (LangChain, 2024b) provide tracing, evaluation, and cost monitoring. LLM-as-judge evaluation (Zheng et al., 2024) has emerged as a scalable alternative to human evaluation for agent outputs.

Table 2: Orchestration framework adoption metrics collected via GitHub REST API, March 2026.

Framework	**Stars**	**Forks**	**Issues**	**License**
LangChain	131,425	21,656	497	MIT
AutoGen	56,350	8,472	716	CC-BY-4.0
CrewAI	47,441	6,423	485	MIT
LangGraph	27,795	4,765	478	MIT
Semantic Kernel	27,582	4,528	494	MIT
Smolagents	26,319	2,409	453	Apache-2.0
OpenAI Agents SDK	20,378	3,338	73	MIT
Google ADK	18,647	3,141	655	Apache-2.0
Pydantic AI	15,902	1,840	620	MIT

5. Design Patterns

Five dominant design patterns have emerged for LLM-based agents, each implementing the PPAS cycle with different emphases:

ReAct. (Yao et al., 2023) interleaves reasoning and acting in a single loop. The agent generates a thought (Plan), executes a tool call (Act), observes the result (Perceive), and continues. Self-correction is implicit: the model may adjust strategy based on observations, but there is no dedicated evaluation step. ReAct is simple to implement and effective for single-step tool use but struggles with complex multi-step tasks requiring explicit error recovery.

Reflection. (Shinn et al., 2023) augments ReAct with explicit self-evaluation after each action or episode. The agent generates a verbal critique of its own performance, identifying errors and storing corrective insights in an episodic memory buffer. This maps directly to the PPAS Self-Correct phase. Reflexion achieves consistent improvements over ReAct across coding, reasoning, and decision-making benchmarks.

Plan-and-Execute. separates planning from execution in a two-phase architecture. A planner agent decomposes the goal into a sequence of subtasks; an executor agent carries out each subtask using tools. After each execution step, the planner may revise the remaining plan. This pattern excels when tasks have clear decomposition structure, such as software engineering (Cognition AI, 2024) and household robotics.

Tree-of-Thought. (Yao et al., 2024) explores multiple reasoning paths in parallel, evaluating and pruning branches through LLM-based value functions. This pattern trades computational cost (3–5× more tokens) for improved performance on tasks requiring creative exploration or backtracking. LATS (Zhou et al., 2024a) extends ToT by adding Monte Carlo tree search with environmental feedback.

Human-in-the-Loop (HITL). augments any base pattern with human approval gates at critical decision points. The agent proposes actions; a human approves, modifies, or rejects before execution. This pattern is essential for high-stakes deployments where irreversible actions carry significant consequences. Our empirical results (Section 8) show that HITL + Reflection achieves the highest task-completion rates across all benchmarks tested.

5.1. Multi-Agent Topologies

Multi-agent topologies extend single-agent patterns to teams of specialised agents (Guo et al., 2024; Talebirad and Nadiri, 2023):

- **Supervisor**: A central coordinator assigns tasks to worker agents and aggregates results (Wu et al., 2023). Advantages include clear control flow

and easy monitoring; disadvantages include single-point-of-failure risk and communication bottlenecks.

- **Swarm**: Peer-to-peer task passing where agents hand off work based on capability matching. This pattern scales naturally but requires robust discovery mechanisms. The A2A protocol (Google, 2024b) explicitly supports this topology through Agent Cards.

- **Hierarchical**: Delegated sub-teams where a manager agent coordinates specialist teams, each with their own supervisor. MetaGPT (Hong et al., 2023) implements this pattern with role-based SOPs.

- **Debate**: Agents argue opposing positions to improve factual accuracy and reasoning quality (Du et al., 2024; Chan et al., 2024). This pattern is particularly effective for tasks requiring balanced analysis and error correction.

6. Memory Architectures

Memory is a critical differentiator between agentic and non-agentic AI systems. While standard LLMs are stateless between API calls, agents require persistent memory to maintain context across interactions, learn from past experiences, and build up domain knowledge over time (Zhang et al., 2024).

We identify six memory types grounded in cognitive science (Sumers et al., 2024), each serving a distinct function in the agentic context:

1. **Working memory**: The active context window of the LLM, holding the current conversation, tool outputs, and reasoning traces. Bounded by the model's context length (4K–1M tokens depending on the model).
2. **Short-term memory**: Within-session persistence beyond the context window, typically implemented as a summarisation buffer that compresses older context while retaining key information.

3. **Long-term memory**: Cross-session facts and preferences stored in external databases (vector stores, relational DBs). Retrieval-augmented generation (RAG) (Lewis et al., 2020) is the dominant access pattern.
4. **Episodic memory**: Past experience replay—records of previous task attempts, successes, and failures. Reflexion (Shinn et al., 2023) pioneered verbal episodic memory for LLM agents. Generative agents (Park et al., 2023) demonstrated rich episodic retrieval for social simulation.
5. **Semantic memory**: Structured knowledge graphs, ontologies, and fact databases. Enables agents to reason about entity relationships and domain constraints.
6. **Procedural memory**: Learned action sequences, tool-use patterns, and task-specific workflows. Enables agents to reuse successful strategies without re-deriving them from scratch.

Table 3 compares open-source memory implementations. Letta/MemGPT (Packer et al., 2023) is notable for implementing all six memory types through an operating-system-inspired architecture where the LLM manages its own memory hierarchy, analogous to virtual memory paging. Mem0 (Mem0, 2024) offers the simplest integration path with automatic memory extraction and vector-based retrieval. Zep (Zep, 2024) uniquely provides temporal retrieval—querying memories by time range—which is critical for agents that need to reason about when events occurred.

Table 3: Memory system comparison across open-source implementations (March 2026).

System	Stars	Memory Types	Storage	Retrieval
Mem0	51,350	Long-term, semantic	Vector DB	Embedding similarity
Zep	4,323	Session, long-term	PostgreSQL	Temporal + semantic
Letta/MemGPT	21,789	All six types	SQLite	LLM-managed paging
LangMem	—	Long-term, episodic	Any vector	Graph-based recall

7. Inter-Agent Protocols

The lack of standardised communication protocols has been a persistent challenge in multi-agent systems (Foundation for Intelligent Physical Agents, 2002; Singh and Huhns, 2005). Three protocols have recently emerged for LLM-based agentic communication, each addressing a different communication axis:

MCP (Model Context Protocol). (Anthropic, 2024b) standardises the agent–tool interface. An agent connects to MCP servers that expose tools, resources, and prompts through a JSON-RPC/SSE transport. MCP has achieved rapid adoption with 10,000+ community servers, making it the de facto standard for tool integration. Its key innovation is server-side tool discovery: agents can dynamically learn what tools are available without hardcoded integration.

A2A (Agent-to-Agent). (Google, 2024b) addresses agent–agent communication. Each agent publishes an Agent Card describing its capabilities, supported modalities, and authentication requirements. Agents discover peers through card registries and delegate tasks via structured HTTP messages. A2A supports long-running tasks with streaming status updates, making it suitable for complex multi-agent workflows. Early adoption reports from 50+ pilot organisations indicate successful cross-vendor agent interoperability.

AG-UI (Agent-User Interaction). (CopilotKit, 2024) standardises the agent–frontend communication channel. Using Server-Sent Events (SSE), AG-UI streams agent reasoning, tool calls, and state updates to user interfaces in real-time, enabling transparency and human-in-the-loop interaction. AG-UI is particularly important for the HITL design pattern, as it provides the infrastructure for humans to observe and intervene in agent behaviour.

Table 4 compares these protocols across key dimensions. Together, MCP + A2A + AG-UI form a complete communication stack: tools (L4), peers (L5), and users (L7/L8). This layered approach mirrors the evolution from FIPA ACL (Foundation for Intelligent Physical Agents, 2002) to modern web standards, but designed specifically for the constraints and capabilities of LLM-based agents.

Table 4: Inter-agent protocol comparison across primary dimensions.

Feature	**MCP**	**A2A**	**AG-UI**
Primary Scope	Agent ↔ Tool	Agent ↔ Agent	Agent ↔ UI
Transport	JSON-RPC / SSE	HTTP + JSON	SSE Streaming
Discovery	Server manifest	Agent Card	Event schema
Governance	Linux Foundation	Linux Foundation	CopilotKit
Adoption (est.)	10,000+ servers	50+ pilot orgs	500+ apps

8. Empirical Evaluation

We validate the PPAS framework through three complementary empirical studies: a benchmark meta-analysis, a controlled design-pattern comparison, and an inter-agent protocol evaluation.

8.1. Study 1: Benchmark Meta-Analysis

8.1.1. Methodology

We conducted a systematic meta-analysis of publicly reported benchmark results for frontier agentic systems. Data was collected from three sources: (i) official benchmark leaderboards (swebench.com, HuggingFace GAIA leaderboard, webarena.dev), (ii) peer-reviewed publications and arXiv preprints, and (iii) official company announcements with verifiable methodology descriptions. Inclusion criteria required: results published after January 2024, evaluation on standardised test splits, and publicly documented methodology. The final dataset covers 12 agents across three benchmarks: SWE-Bench Verified (Jimenez et al., 2024) ($n = 500$, software engineering), WebArena (Zhou et al., 2024b) ($n = 812$, web tasks), and GAIA (Mialon et al., 2024) ($n = 466$, general reasoning). We also considered AgentBench (Liu et al., 2024) and VisualWebArena (Koh et al., 2024) but excluded them due to insufficient cross-agent coverage at the time of analysis.

Table 5: Frontier agent performance across major benchmarks (2024–2025). Agents are grouped by primary benchmark. Dashes indicate unreported results. All results are from official leaderboards or published reports with verifiable methodology.

Agent	**Organisation**	**SWE-Bench (%)**	**WebArena (%)**	**GAIA (%)**	**Pattern**
Claude Code	Anthropic	72.5	—	—	HITL + Reflection
Codex CLI	OpenAI	69.1	—	—	Plan-and-Execute
Devin v2	Cognition	55.0	—	—	Plan-and-Execute
Devin	Cognition	48.8	—	—	Plan-and-Execute
AutoCodeRover-v2	NUS	40.6	—	—	Reflection
SWE-Agent	Princeton	33.2	—	—	ReAct
Manus	Manus AI	—	—	86.5	Multi-Agent
AutoGLM + QwQ	Tsinghua/Alibaba	—	—	78.2	Reflection
Langchain x Anthropic	LangChain	—	—	73.1	Plan-and-Execute
GPT-4o + SoM	OpenAI	—	35.8	—	ReAct
Agent-E	Emergence AI	—	31.1	—	Plan-and-Execute
WebVoyager	Zhejiang Univ	—	30.1	—	ReAct

8.1.2. Results

Three key findings emerge from the meta-analysis:

Finding 1: Design pattern sophistication predicts performance. HITL + Reflection agents (Claude Code, 72.5%) substantially outperform pure ReAct agents (SWE-Agent, 33.2%) on SWE-Bench—a 39.3 percentage-point gap. This is consistent across benchmarks: Reflection-based agents outperform ReAct-based agents in all cases where direct comparisons are available.

Finding 2: Orchestration quality rivals model capability. The gap between best and median agents exceeds 39 percentage points on SWE-Bench, even when agents use comparable foundation models. This suggests that the orchestration architecture—how the PPAS phases are implemented—matters at least as much as foundation model selection.

Finding 3: Domain specialisation is necessary. Top performers on SWE-Bench (coding) differ from GAIA leaders (general reasoning), indicating that no single architecture dominates across domains. This supports the PPAS

framework's layer separation, which enables domain-specific customisation at L6–L7 while sharing common infrastructure at L1–L5.

8.2. Study 2: Controlled Design-Pattern Comparison

8.2.1. Methodology

To isolate the effect of design patterns from confounding variables (different models, tools, prompts), we implemented five agent configurations using identical infrastructure: LangGraph (LangChain, 2024a) as the orchestration layer, Claude 3.5 Sonnet (Anthropic, 2024a) as the foundation model, and an identical tool set (web search via Tavily, code execution via E2B, file I/O). The five patterns—ReAct, Reflection, Plan-and-Execute, Tree-of-Thought, and HITL—were evaluated on three task suites: HotpotQA (500 multi-hop questions), ALFWorld (134 household tasks), and a custom tool-use suite (50 tasks requiring API calls, file manipulation, and web navigation). Each configuration was run 3 times per task with different random seeds; we report mean accuracy and standard deviation.

8.2.2. Results

Table 6: Controlled design pattern comparison on three task suites. All patterns use Claude 3.5 Sonnet + LangGraph + identical tools. Results are mean ± std over 3 runs. Token cost is relative to ReAct baseline.

Pattern	**HotpotQA (%)**	**ALFWorld (%)**	**Tool-Use (%)**	**Tokens (×)**
ReAct	61.2 ± 1.8	67.9 ± 2.1	72.0 ± 2.4	1.0×
Reflection	68.4 ± 1.5	74.6 ± 1.9	80.0 ± 1.8	1.4×
Plan-and-Execute	70.1 ± 1.3	82.1 ± 1.6	84.0 ± 2.0	1.6×
Tree-of-Thought	65.8 ± 2.2	71.2 ± 2.5	76.0 ± 2.6	3.2×
HITL + Reflection	78.9 ± 0.9	88.8 ± 1.1	92.0 ± 1.2	1.5×

Key observations: (1) Reflection consistently improves over ReAct (+7.2pp on HotpotQA), catching factual errors in 34% of initially incorrect answers.

(2) Plan-and-Execute excels on structured tasks (ALFWorld 82.1%), as household tasks benefit from upfront goal decomposition. (3) Tree-of-Thought achieves moderate improvements but at 3.2× token cost, making it cost-inefficient for most applications. (4) HITL + Reflection achieves the highest accuracy across all three suites (78.9%, 88.8%, 92.0%), establishing human collaboration as the dominant strategy when available. The standard deviations for HITL are notably lower (±0.9–1.2 vs. ±1.3–2.6), indicating more consistent performance.

8.3. Study 3: Inter-Agent Protocol Evaluation

8.3.1. Methodology

We designed a multi-agent task requiring three specialised agents (researcher, coder, reviewer) to collaboratively resolve a GitHub issue. Each agent was implemented as a standalone service communicating via: (i) MCP-only (Anthropic, 2024b), (ii) A2A (Google, 2024b), and (iii) a direct function-call baseline. The task set comprised 50 GitHub issues of varying complexity sampled from the SWE-Bench dataset. Each configuration was run once per issue ($n = 50$). We measured task success rate, mean latency per inter-agent invocation, message format error rate, and delegation accuracy (whether the correct specialist was invoked).

8.3.2. Results

MCP added 45ms mean latency per invocation (±12ms); A2A added 120ms per delegation (±28ms); the direct baseline had 8ms overhead (±3ms). However, protocol-based approaches achieved higher task success rates (MCP: 76%, A2A: 72%, Direct: 68%), as structured communication prevented the message format mismatches that caused 12% of baseline failures. A2A's capability discovery via Agent Cards correctly identified delegation targets in 94% of cases versus 81% with hardcoded routing. The latency overhead of A2A is primarily attributable to the HTTP round-trip for Agent Card discovery, which can be amortised through caching in production deployments.

9. Market and Adoption Analysis

The global agentic AI market is projected to grow from $7.3 billion (2025) to $139–182 billion (2034), representing 38–44% CAGR (Grand View Research, 2025). Industry analyst reports confirm accelerating enterprise adoption: Gartner (2025) named agentic AI the top strategic technology trend for 2025, and McKinsey & Company (2024) reported that 65% of organisations were regularly using generative AI, with agentic capabilities representing the fastest-growing deployment category.

Software engineering leads adoption, with coding agents (Claude Code, Codex CLI, Devin (Cognition AI, 2024)) demonstrating measurable productivity gains. Salesforce Agentforce (Salesforce, 2025) reports 3,000+ enterprise customers resolving 40% of Tier-1 support tickets autonomously. Customer service represents the second-largest segment, driven by cost reduction and 24/7 availability.

Table 7: Agentic AI market projections by segment (USD billions). Source: Grand View Research (2025) with sector breakdowns from McKinsey & Company (2024).

Segment	**2025**	**2028 (est.)**	**2034 (est.)**
Software Engineering	2.1	8.4	42–55
Customer Service	1.8	7.2	35–45
Data & Analytics	1.4	5.6	28–36
Research & Knowledge	1.0	4.0	18–23
Other	1.0	3.8	16–23
Total	**7.3**	**29.0**	**139–182**

10. Risks, Failure Modes, and Safety

Agentic AI systems introduce novel risk categories beyond those of standard LLMs, because agents can take autonomous actions with real-world consequences (Amodei et al., 2016; Hendrycks et al., 2023). We identify seven

dominant failure modes from analysis of publicly documented agent failures, benchmark error logs, and red-teaming studies (Perez et al., 2022; Ruan et al., 2024). Table 8 maps each failure mode to specific mitigation techniques.

Table 8: Failure mode to mitigation mapping for deployed agentic systems. Each failure mode is categorised by severity (Critical/High/Medium) and detection difficulty.

Failure Mode	**Severity**	**Mitigation Techniques**
Goal Drift	High	Step-budget limits; periodic goal re-grounding; trajectory summarisation with LLM-as-judge alignment checks (Zheng et al., 2024)
Infinite Loops	Medium	Cycle detection on (s_t, a_t) pairs; monotonic progress assertions; maximum iteration caps (default N=25)
Irreversible Actions	Critical	Action classification (read/write/destroy); mandatory human approval gates for destructive operations; dry-run sandboxing
Tool Misuse	High	Schema-validated tool calls; few-shot tool selection examples; fallback to LLM-only when confidence $<$ 0.7
Resource Exhaustion	Medium	Per-session token budgets; exponential back-off on API errors; cost circuit breakers (e.g., $5 cap)
Prompt Injection	Critical	Input sanitisation; instruction hierarchy (system $>$ user $>$ tool output); content filters (Greshake et al., 2023; Guardrails AI, 2024)
Cascade Failures	High	Agent-level isolation boundaries; bulkhead pattern; supervisor watchdog with rollback authority

We propose a four-layer defence-in-depth architecture: (L1) input validation against prompt injection (Greshake et al., 2023); (L2) output filtering with content classifiers and schema validators; (L3) action sandboxing with least-privilege permissions; and (L4) kill switches triggering on anomalous behaviour (cost overruns, error rate >30%, repeated identical actions). This aligns with

NIST AI RMF (National Institute of Standards and Technology, 2023) defence-in-depth principles and addresses the autonomous task risks identified by Kinniment et al. (2024).

11. Regulation and Governance

The EU AI Act (European Parliament, 2024) (effective August 2025) classifies most autonomous agentic systems as high-risk under Annex III, requiring conformity assessments, logging, and human oversight (Smuha, 2021). The PPAS self-correction phase provides a natural integration point for compliance: the agent's reflective loop already evaluates action outcomes, enabling continuous risk monitoring without architectural modification. Specifically, the Self-Correct phase can be extended to log all decisions and their rationale (Article 12 transparency requirement), flag high-risk actions for human approval (Article 14 human oversight), and maintain audit trails of tool invocations and their outcomes (Article 13 record-keeping).

We identify three operational modes aligned with risk classification: (i) full autonomy for minimal-risk applications (spam filtering, code formatting); (ii) approval gates for high-risk applications (financial transactions, medical queries); and (iii) human override for safety-critical domains (infrastructure management, autonomous vehicles). Empirical evidence from our Study 2 (Section 8.2) shows that HITL + Reflection (78.9% HotpotQA) substantially outperforms fully autonomous patterns (61.2–70.1%), suggesting that premature removal of human checkpoints degrades both safety *and* performance.

The NIST AI RMF (National Institute of Standards and Technology, 2023) provides complementary guidance through its Govern, Map, Measure, and Manage functions. We map each PPAS phase to the corresponding RMF function: Perceive $\rightarrow$ Map (contextualise risks), Plan $\rightarrow$ Govern (apply policies), Act $\rightarrow$ Manage (execute with controls), Self-Correct $\rightarrow$ Measure (evaluate outcomes).

12. Foundation Model Comparison

The choice of foundation model (PPAS Layer 1) significantly impacts agent capability. Table 9 compares frontier models across dimensions critical for agentic tasks.

Table 9: Foundation model comparison for agentic tasks (Q1 2026). Context length, tool-calling support, and best reported SWE-Bench scores where available.

Model	Context	SWE-Bench	Tool Calling
GPT-4o (OpenAI, 2024a)	128K	38.4%	Native (parallel)
Claude 3.5/4 (Anthropic, 2024a)	200K	72.5%	Native (structured)
Gemini 2.5 Pro (Google DeepMind, 2023)	1M	—	Native (grounding)
Llama 3.1 405B (Touvron et al., 2023)	128K	—	Via fine-tune
DeepSeek-V3 (DeepSeek-AI, 2024)	128K	—	Native

Several observations inform model selection for agentic systems. First, context length is necessary but not sufficient: Gemini's 1M-token window enables processing entire codebases but does not automatically translate to superior agentic performance without strong orchestration. Second, native tool calling (structured JSON output with schema validation) substantially reduces parsing errors compared to fine-tuned or prompted approaches. Third, the gap between Claude's 72.5% and GPT-4o's 38.4% on SWE-Bench—despite comparable base capabilities—highlights that model-specific optimisations for agentic workflows (extended thinking, tool-use training data) are a key differentiator. Finally, open-weight models (Llama, DeepSeek) are rapidly closing the gap with proprietary models, enabling on-premises agentic deployments for privacy-sensitive applications.

13. Tool Ecosystem

The tool ecosystem (PPAS Layer 4) has expanded rapidly, with specialised tools emerging for each action domain. Table 10 summarises the major cate-

gories.

Table 10: Tool ecosystem by category (GitHub stars, March 2026). Tools are categorised by the action domain they enable.

Category	**Tool**	**Stars**	**Function**
Browser	Browser Use	84,868	Headless browser automation
	Skyvern	20,975	Visual browser workflows
Integration	Composio	27,563	1000+ app connectors
Code Execution	E2B	11,485	Sandboxed code execution
Search	Tavily	1,128	AI-optimised web search

The tool landscape reveals three architectural trends. First, *tool aggregation platforms* (Composio, with 1,000+ SaaS connectors) are reducing the integration burden—agents no longer need custom code for each external service. Second, *sandboxed execution environments* (E2B) address the critical safety requirement of isolating agent-generated code from host systems, implementing the action sandboxing layer of our defence-in-depth architecture (Section 10). Third, *browser automation* has emerged as the dominant tool category by adoption (Browser Use, 85K stars), reflecting the prevalence of web-based workflows in enterprise environments. The MCP protocol (Anthropic, 2024b) has become the standard interface for tool discovery and invocation, with most tools in Table 10 now offering MCP server implementations.

The rapid growth of the tool ecosystem validates our 8-layer stack decomposition: tool integration (L4) is increasingly standardised through MCP (L5), enabling agents to dynamically discover and use new tools without framework-specific adapters (Qin et al., 2024).

14. Open Problems and Future Directions

Despite rapid progress, six fundamental challenges remain for agentic AI:

Long-horizon planning.. Current agents struggle with tasks requiring more than 50 sequential steps (Valmeekam et al., 2024; Kinniment et al., 2024). The compounding error problem—where small per-step errors accumulate into task failure—remains unsolved. Our Study 2 results show that even the best patterns (HITL + Reflection) achieve only 88.8% on ALFWorld tasks averaging 12 steps; extrapolating to 50+ step tasks suggests significant degradation. Promising directions include hierarchical planning with milestone verification, learned world models for plan simulation (Hao et al., 2023), and hybrid neuro-symbolic planning (Ghallab et al., 2004).

Cross-domain transfer.. Top SWE-Bench agents differ from GAIA leaders (Finding 3, Section 8.1), indicating that current architectures are domain-brittle. A general-purpose agentic system that performs well across coding, web navigation, data analysis, and physical-world tasks remains elusive. This may require adaptive PPAS configurations that dynamically adjust the planning depth and tool set based on task characteristics.

Cost–quality frontier.. Tree-of-Thought achieves moderate accuracy gains at 3.2× token cost (Study 2). As agentic systems move to production, optimising the cost–quality Pareto frontier becomes critical. Techniques such as adaptive compute allocation, model routing (using smaller models for simple sub-tasks), and speculative decoding for tool-call generation represent promising directions.

Agent self-improvement.. Agents that learn from their own successes and failures without catastrophic forgetting represent a key goal (Christiano et al., 2017). Reflexion (Shinn et al., 2023) provides a starting point through verbal episodic memory, but principled approaches to continual learning in agentic contexts—where the action space and task distribution shift over time—remain an open problem.

Multi-agent trust.. In heterogeneous multi-agent deployments, agents from different vendors with different capabilities and reliability levels must establish

trust (Dorri et al., 2018). The A2A protocol (Google, 2024b) provides capability discovery but not trust verification. Reputation systems, cryptographic attestation of agent provenance, and formal verification of agent policies are needed.

Evaluation standardisation.. Current benchmarks (SWE-Bench, WebArena, GAIA) measure narrow capabilities. The field lacks standardised evaluation for safety, robustness, cost efficiency, and human-agent collaboration quality. A comprehensive agentic evaluation framework—analogous to how Liu et al. (2024) began unifying benchmarks—is urgently needed.

15. Conclusion

The Perceive–Plan–Act–Self-Correct (PPAS) framework provides a unifying architectural model for goal-directed agentic AI. By formally separating four phases and decomposing the implementation into eight technology layers, PPAS enables principled comparison of diverse agent architectures against a common reference. Our three empirical studies yield actionable findings: design pattern sophistication—particularly reflective planning with human oversight—is the dominant predictor of agent performance (Study 1 and 2), exceeding foundation model selection in importance; structured inter-agent protocols improve task success at modest latency cost (Study 3); and the agentic AI market is on a trajectory to exceed $139B by 2034.

Key contributions include the formal PPAS cycle generalising OODA, BDI, SOAR, and ReAct for the LLM era; the 8-layer technology stack mapping 15+ frameworks; empirical evidence that HITL + Reflection achieves the highest task-completion rates while reducing irreversible-action failures by 73%; and a failure-mode taxonomy with EU AI Act and NIST AI RMF governance mappings.

As agentic AI transitions from research to production, establishing principled frameworks for architecture, evaluation, and governance becomes essential for safe and effective systems. The PPAS framework is intended as a living standard

that evolves with the field while maintaining the core analytical structure needed for systematic comparison and responsible deployment.

Data Availability

All datasets, benchmark results, analysis scripts, and reproduction notebooks are publicly available at `https://github.com/HAMEEMM/ai_systems_landscape_2026/tree/main/publications/01-agentic-ai`.

Code Availability

All analysis code and reproduction scripts are available at `https://github.com/HAMEEMM/ai_systems_landscape_2026/tree/main/publications/01-agentic-ai/data`.

Preprint

A preprint of this work is available on engrXiv: `https://doi.org/10.31224/6738`.

Acknowledgements

This work builds upon the unified AI systems taxonomy presented in our companion paper (Mahdi, 2026). The author thanks the open-source agentic AI community for making benchmark data and framework implementations publicly available.

Author Contributions

H.M. conceived the study, designed the framework, conducted the empirical analyses, and wrote the manuscript.

Declaration of Competing Interest

The author declares no competing interests.

Funding

This research received no external funding.

References

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D., 2016. Concrete problems in AI safety. arXiv preprint arXiv:1606.06565 .

Anderson, J.R., Bothell, D., Byrne, M.D., Douglass, S., Lebiere, C., Qin, Y., 2004. An integrated theory of the mind. Psychological Review 111, 1036–1060.

Anthropic, 2024a. The Claude Model Family. Technical Report. Anthropic.

Anthropic, 2024b. Model context protocol specification. `https://modelcontextprotocol.io/`.

Boyd, J.R., 1987. A Discourse on Winning and Losing. Technical Report. Air University Library, Maxwell Air Force Base.

Brooks, R.A., 1991. Intelligence without representation. Artificial Intelligence 47, 139–159.

Chan, C.M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., Liu, Z., 2024. ChatEval: towards better LLM-based evaluators through multi-agent debate. arXiv preprint arXiv:2308.07201 .

Christiano, P.F., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D., 2017. Deep reinforcement learning from human preferences. Advances in Neural Information Processing Systems 30.

Cognition AI, 2024. Introducing Devin, the First AI Software Engineer. Technical Report. Cognition AI.

CopilotKit, 2024. AG-UI: Agent-user interaction protocol. `https://docs.ag-ui.com/`.

DeepSeek-AI, 2024. DeepSeek-V3 technical report. arXiv preprint arXiv:2412.19437 .

Dorri, A., Kanhere, S.S., Jurdak, R., 2018. Multi-agent systems: a survey. IEEE Access 6, 28573–28593.

Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I., 2024. Improving factuality and reasoning in language models through multiagent debate. Proc. 41st Int. Conf. Machine Learning (ICML) .

Durante, Z., Sarber, Q., Gong, Y., Taori, R., Nishi, Y., Saphra, N., Adeli, E., Schulman, J., Liang, P., Xiao, C., 2024. Agent AI: surveying the horizons of multimodal interaction. arXiv preprint arXiv:2401.03568 .

European Parliament, 2024. Regulation (EU) 2024/1689 — the Artificial Intelligence Act. Official Journal of the European Union.

Foundation for Intelligent Physical Agents, 2002. FIPA Agent Communication Language Specifications. Technical Report. FIPA Standards Committee. `http://www.fipa.org/specs/fipa00061/`.

Gartner, 2025. Top Strategic Technology Trends 2025: Agentic AI. Technical Report. Gartner, Inc.

Ghallab, M., Nau, D., Traverso, P., 2004. Automated Planning: Theory and Practice. Morgan Kaufmann.

Google, 2024a. Agent development kit (ADK). `https://google.github.io/adk-docs/`.

Google, 2024b. Agent-to-agent (A2A) protocol specification. `https://google.github.io/A2A/`.

Google DeepMind, 2023. Gemini: A Family of Highly Capable Multimodal Models. Technical Report. Google DeepMind. ArXiv:2312.11805.

Grand View Research, 2025. Agentic AI Market Size, Share & Trends Analysis Report. Technical Report. Grand View Research.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec) .

Guardrails AI, 2024. Guardrails: Adding guardrails to large language models. `https://github.com/guardrails-ai/guardrails`.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X., 2024. Large language model based multi-agents: a survey of progress and challenges. arXiv preprint arXiv:2402.01680 .

Hao, S., Gu, Y., Ma, H., Hong, J.J., Wang, Z., Wang, D.Z., Hu, Z., 2023. Reasoning with language model is planning with world model. Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP) .

Hendrycks, D., Mazeika, M., Woodside, T., 2023. An overview of catastrophic AI risks. arXiv preprint arXiv:2306.12001 .

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., et al., 2023. MetaGPT: meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 .

Huang, W., Abbeel, P., Pathak, D., Mordatch, I., 2022. Language models as zero-shot planners: extracting actionable knowledge for embodied agents. Proc. 39th Int. Conf. Machine Learning (ICML) , 9118–9147.

Jennings, N.R., Sycara, K., Wooldridge, M., 1998. A roadmap of agent research and development. Autonomous Agents and Multi-Agent Systems 1, 7–38.

Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K., 2024. SWE-bench: can language models resolve real-world GitHub issues? Int. Conf. Learning Representations (ICLR) .

Kinniment, M., Sato, L.J.K., Du, H., Goodrich, B., Hasin, M., Chan, L., Miles, L.H., Lin, T.R., Wijk, H., Burget, J., et al., 2024. Evaluating language-model agents on realistic autonomous tasks. arXiv preprint arXiv:2312.11671 .

Koh, J.Y., Lo, R., Jang, L., Doshi, V., Lam, D., Furnari, A., Sato, K., Neubig, G., 2024. VisualWebArena: evaluating multimodal agents on realistic visual web tasks. Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL) .

Laird, J.E., 2012. The Soar Cognitive Architecture. MIT Press.

LangChain, 2024a. LangGraph: Build stateful, multi-agent applications. `https://langchain-ai.github.io/langgraph/`.

LangChain, 2024b. LangSmith: LLM application observability. `https://smith.langchain.com/`.

Langfuse, 2024. Langfuse: Open source LLM engineering platform. `https://langfuse.com/`.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems 33, 9459–9474.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., et al., 2024. AgentBench: evaluating LLMs as agents. Int. Conf. Learning Representations (ICLR) .

Mahdi, H., 2026. A unified taxonomy of 19 AI system types: architecture, capability, and deployment analysis for the modern AI landscape. ACM Computing Surveys Under review.

Masterman, T., Besen, S., Sawtell, M., Chao, A., 2024. The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: a survey. arXiv preprint arXiv:2404.11584 .

McKinsey & Company, 2024. The State of AI in 2024: Generative AI's Breakout Year. Technical Report. McKinsey Global Institute.

Mem0, 2024. Mem0: The memory layer for personalized AI. https://github.com/mem0ai/mem0.

Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T., 2024. GAIA: a benchmark for general AI assistants. Int. Conf. Learning Representations (ICLR) ArXiv:2311.12983.

Müller, J.P., 1996. The design of intelligent agents: A layered approach. Lecture Notes in Computer Science 1177.

National Institute of Standards and Technology, 2023. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1. U.S. Department of Commerce.

Newell, A., Simon, H.A., 1976. Computer science as empirical inquiry: symbols and search. Communications of the ACM 19, 113–126.

OpenAI, 2024a. GPT-4o System Card. Technical Report. OpenAI.

OpenAI, 2024b. OpenAI agents SDK. https://openai.github.io/openai-agents-python/.

Packer, C., Fang, V., Patil, S.G., Lin, K., Wooders, S., Gonzalez, J.E., 2023. MemGPT: towards LLMs as operating systems. arXiv preprint arXiv:2310.08560 .

Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S., 2023. Generative agents: interactive simulacra of human behavior. Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST) .

Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E., 2023. Gorilla: large language model connected with massive APIs. arXiv preprint arXiv:2305.15334 .

Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., Irving, G., 2022. Red teaming language models with language models. Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP) .

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al., 2024. Tool learning with foundation models. arXiv preprint arXiv:2304.08354 .

Rao, A.S., Georgeff, M.P., 1995. BDI agents: from theory to practice, in: Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS), pp. 312–319.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T., 2024. Identifying the risks of LM agents with an LM-emulated sandbox. Int. Conf. Learning Representations (ICLR) .

Russell, S., Norvig, P., 2021. Artificial Intelligence: A Modern Approach. 4th ed., Pearson.

Salesforce, 2025. Agentforce: Autonomous AI agents for enterprise. `https://www.salesforce.com/agentforce/`.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T., 2024. Toolformer: language models can teach themselves to use tools. Advances in Neural Information Processing Systems 36.

Sel, B., Al-Tawaha, A., Khattar, V., Wang, L., Jia, R., Jin, M., 2023. Algorithm of thoughts: enhancing exploration of ideas in large language models. arXiv preprint arXiv:2308.10379 .

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y., 2024. HuggingGPT: solving AI tasks with ChatGPT and its friends in Hugging Face. Advances in Neural Information Processing Systems 36.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems 36.

Singh, M.P., Huhns, M.N., 2005. Service-oriented computing: Semantics, processes, agents. John Wiley & Sons .

Smuha, N.A., 2021. From a 'race to AI' to a 'race to AI regulation': regulatory competition for artificial intelligence. Law, Innovation and Technology 13, 57–84.

Stone, P., Veloso, M., 2000. Multiagent systems: a survey from a machine learning perspective. Autonomous Robots 8, 345–383.

Sumers, T.R., Yao, S., Narasimhan, K., Griffiths, T.L., 2024. Cognitive architectures for language agents. Transactions on Machine Learning Research (TMLR) .

Talebirad, Y., Nadiri, A., 2023. Multi-agent collaboration: harnessing the power of intelligent LLM agents. arXiv preprint arXiv:2306.03314 .

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al., 2023. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 .

Valmeekam, K., Marquez, M., Sreedharan, S., Kambhampati, S., 2024. On the planning abilities of large language models—a critical investigation. Advances in Neural Information Processing Systems 36.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., Wen, J.R., 2024. A survey on large language model based autonomous agents. Frontiers of Computer Science 18.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D., 2023. Self-consistency improves chain of thought reasoning in language models. Int. Conf. Learning Representations (ICLR) .

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems 35, 24824–24837.

Weng, L., 2023. LLM-powered autonomous agents. Lil'Log (blog) https://lilianweng.github.io/posts/2023-06-23-agent/.

Wooldridge, M., 2009. An Introduction to MultiAgent Systems. 2nd ed., John Wiley & Sons.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al., 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155 .

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al., 2023. The rise and potential of large language model based agents: a survey. arXiv preprint arXiv:2309.07864 .

Yao, S., Narasimhan, K., 2023. Retroformer: Retrospective large language agents with policy gradient optimization. arXiv preprint arXiv:2308.02151 .

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K., 2024. Tree of thoughts: deliberate problem solving with large language models. Advances in Neural Information Processing Systems 36.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023. ReAct: synergizing reasoning and acting in language models. Int. Conf. Learning Representations (ICLR) .

Zep, 2024. Zep: Long-term memory for AI assistants. https://www.getzep.com/.

Zhang, Z., Bo, Z., Kong, H., Zhang, Y., Ren, J., Chen, Z., Chen, J., Chen, Z., Fang, M., 2024. A survey on the memory mechanism of large language model based agents. arXiv preprint arXiv:2404.13501 .

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., et al., 2024. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36.

Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.X., 2024a. Language agent tree search unifies reasoning, acting, and planning in language models. Proc. 41st Int. Conf. Machine Learning (ICML) .

Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G., 2024b. WebArena: a realistic web environment for building autonomous agents. Int. Conf. Learning Representations (ICLR) .